# vasp_diagnose.py

**Find out why VASP runs failed, fix them and resubmit them**

**Part of VASP Workflow · SLURM clusters and local machines**

Keivan Esfarjani

October 2026 | https://github.com/KeivanS/VASP-Flow

## Contents


**1 What it is for** **2**

**2 Quick start** **2**

**3 A worked example** **2**
- 3.1 The list . . . 2
- 3.2 Step 1: diagnose . . . 3
- 3.3 Step 2: look at the result . . . 3
- 3.4 Step 3: submit . . . 4
- 3.5 Step 4: check again . . . 4
- 3.6 The material it could not diagnose . . . 4

**4 Common situations** **5**

**5 Reference** **5**
- 5.1 Command line . . . 5
- 5.2 The list file . . . 6
- 5.3 Kinds of runs . . . 6
- 5.4 When is a step OK? . . . 7
- 5.5 Failure causes and their repairs . . . 8
- 5.6 What is redone . . . 8
- 5.7 Order of the resubmitted jobs . . . 9
- 5.8 Output files . . . 9
- 5.9 Limitations . . . 10

## 1 What it is for

After a batch of VASP-Flow calculations — one project or a few thousand materials — some runs will not have finished: a job ran out of memory or time, an SCF did not converge, a band structure diverged, LOBSTER stopped, a convergence test broke off. `vasp_diagnose.py` finds these runs, tells you why each one failed and writes, for each of them, a shell script that repairs the inputs and resubmits exactly what has to be redone. One more script submits all of these repairs.

It checks the steps `00_convergence`, `01_relax`, `02_scf`, `03_bands`, `04_dos`, `06_dfpt` and `08_lobster`, for runs made with `vasp-agent.py`, `vasp-agent-slurm.py` or the high-throughput drivers.

> **Tip.** Running `vasp_diagnose.py` never changes or submits anything. It only writes a report and scripts. Your runs are touched only when you run those scripts.

## 2 Quick start

On the cluster, in your VASP-Flow folder, give it the list file you made the runs from:

```
cd ~/VASP-Flow
python3 vasp_diagnose.py my_list.txt                    # 1. diagnose
    (read-only)
less vasp_diagnose_*/diagnosis.tsv                      # 2. look at
    the result
nohup bash vasp_diagnose_*/submit_all_fixes.sh > fixes.log 2>&1 &
    # 3. resubmit
```

That is all. `my_list.txt` stands for your own list, e.g. `semimetals_list.txt`: the file of material names that the runs were created from, one per line. A name is whatever the material's folder is called — a Materials Project ID (`mp-149`), a formula (`GaAs`), `GaAs_PBE`, ... The script finds each material's folder by itself. When the resubmitted jobs have finished, run step 1 again to check that everything is now fine.

> **Note.** Run it when your jobs have left the queue. Materials that still have jobs queued or running are skipped (shown as `queued`).

## 3 A worked example

Say you created 300 materials with a high-throughput driver from `~/VASP-Flow/semimetals_list.txt`, into the run folder `~/VASP-Flow/HT_semimetals/`, and the queue is now empty.

### 3.1 The list

The list is the file you already have, for example

```
# ~/VASP-Flow/semimetals_list.txt
mp-149
mp-2534
mp-20351
Bi2Se3
GaAs_PBE
...
```

Each name is looked up in the folder that holds the list file, so `mp-149` is found as `~/VASP-Flow/HT_semimetals/materials/mp-149`, and `Bi2Se3` as `~/VASP-Flow/HT_semimetals/materials/Bi2Se3` or as a project folder `~/VASP-Flow/Bi2Se3`. Names must match the folder names exactly (`GaAs` does not match `GaAs_PBE`). You can also list folder paths, or mix names and paths (section 5.2).

### 3.2 Step 1: diagnose

```
cd ~/VASP-Flow
python3 vasp_diagnose.py semimetals_list.txt
```

It prints a summary — how many materials are fine and how many failed for which reason:

```
Folders checked: 300   (layouts: ht 300)
  ok              262
  oom              14
  scf_nelm          6
  timeout           5
  spilling_high     5
  lobster_fail      4
  nscf_diverged     2
  missing           1
  unknown           1
Fix scripts: 31 to submit, 1 to review (unknown cause)
  review:
     .../vasp_diagnose_20261012_1530/fix/HT_semimetals__mp-20351.sh
Report : .../vasp_diagnose_20261012_1530/diagnosis.tsv
Submit : nohup bash .../submit_all_fixes.sh >
    .../submit_all_fixes.log 2>&1 &
```

Here 262 materials are fine. 31 failed for a known reason and have a fix script ready. One failed for a reason the tool does not recognise; its script is kept back for you to look at. 5 finished but have a high LOBSTER charge spilling (reported only), and one name of the list has no folder at all (`missing`). Section 5.5 explains each name.

Everything is written into a new folder `vasp_diagnose_<date>_<time>/`:

```
vasp_diagnose_20261012_1530/
  diagnosis.tsv          the report: one line per material
  fix/<name>.sh          one repair script per failed material
  submit_all_fixes.sh    runs all the repair scripts
```

### 3.3 Step 2: look at the result

`diagnosis.tsv` has one line per material: the status of every step and what will be done:

```
label                  ...  steps
                                                  action
HT_semimetals__mp-149       relax:ok scf:ok bands:ok lobster:ok
              ok
HT_semimetals__mp-2534      relax:ok scf:oom bands:not_run
   lobster:not_run oom -> scf>bands>lobster
HT_semimetals__mp-8062      relax:ok scf:scf_nelm bands:ok lobster:ok
       scf_nelm -> scf>bands>lobster
HT_semimetals__mp-20351     relax:unknown scf:ok ...
                        REVIEW unknown -> relax>scf>bands>lobster
```

Read it with `less -S`, `column -t -s$'\t '`, or a spreadsheet. Then open one or two repair scripts to see what they will do, e.g.

```
less vasp_diagnose_20261012_1530/fix/HT_semimetals__mp-2534.sh
```

The top of each script says what was found, including the error lines from the logs:

```
# vasp_diagnose fix for HT_semimetals__mp-2534
# folder : /home/me/VASP-Flow/HT_semimetals/materials/mp-2534
    (layout: ht)
# diagnosed 2026-10-12 15:30:  relax: ok  scf: oom  bands: not_run
    lobster: not_run
#   scf | forrtl: severe (41): insufficient virtual memory
# resubmits: scf -> bands -> lobster
```

and further down you see the changes and the submission:

```
set_tag 02_scf/INCAR KPAR 8                  # KPAR was 40
stash 02_scf                                 # old outputs ->
    stale_<date>/02_scf/
...
J_scf=$(sbatch --parsable --job-name=mp-2534_scf --time=00:30:00
    --mem=0 job.sbatch scf)
J_bands=$(sbatch ... --dependency=afterok:$J_scf job.sbatch bands)
J_lobster=$(sbatch ... --dependency=afterany:$J_bands job.sbatch
    lobster)
```

You may edit a script before submitting it (another walltime, another KPAR, ...).

### 3.4 Step 3: submit

```
nohup bash vasp_diagnose_20261012_1530/submit_all_fixes.sh >
    fixes.log 2>&1 &
tail -f fixes.log
```

`fixes.log` shows one block per material:

```
== HT_semimetals__mp-2534
  scf: KPAR 40 -> 8 (40 ranks)
  submitted scf = 4512301 (00:30:00)
  submitted bands = 4512302 (00:30:00)
  submitted lobster = 4512303 (03:00:00)
```

`nohup` lets it keep running after you log out. It never puts more than 200 of your jobs in the queue at once (it waits until there is room). If it is interrupted, start it again: materials it already submitted are skipped.

### 3.5 Step 4: check again

When the queue is empty again, diagnose once more, into a new folder:

```
python3 vasp_diagnose.py semimetals_list.txt --out diagnose_round2
```

The repaired materials now show `ok`. The ones that failed again get a new diagnosis and a new script. Repeat until only `ok` (and cases you accept, such as `spilling_high`) are left.

### 3.6 The material it could not diagnose

For `unknown` cases, read the error lines at the top of the script. Then either repair the run by hand, or let the script run as it is: open `submit_all_fixes.sh`, where these scripts appear at the end as commented-out lines, and remove the `#`. To submit all unknown cases without editing, diagnose again with `--retry-unknown`.

## 4 Common situations

**Only some runs, or a single project.** Make a short list with just those material names, or give a folder:

```
echo GaAs_test > one.txt;  python3 vasp_diagnose.py one.txt
echo HT_semimetals > run.txt; python3 vasp_diagnose.py run.txt   #
    all materials of that run
```

**Runs moved to another disk (e.g. scratch).** If a run folder holds a `MOVED_TO_SCRATCH.txt` note (written by `ht_tools/resume_in_scratch.sh`), its new place is used automatically. Otherwise tell the script where to look:

```
python3 vasp_diagnose.py semimetals_list.txt --root
    /scratch/$USER/VASP-Flow
```

**The same name in two runs.** If a name is found in more than one run folder, all copies are checked and a line `!! GaAs found in 2 runs` is printed. To check only one of them, put its path in the list instead of the name.

**Projects with convergence tests.** For a project made in two phases (convergence tests first, then `submit_calculations.sh` or `run_calculations.sh`), broken ENCUT/k-point tests are repaired and resubmitted; the production steps are *not* started for you, since you choose their ENCUT and k-mesh from the tests. Such a project shows `phase2_pending` once the tests are complete.

**Local projects (no SLURM).** Their repair script runs VASP directly, one step after the other. Run that script on a machine where VASP may run (not on a login node): `bash vasp_diagnose_*/fix/projects__GaAs_test.sh`.

**Fewer jobs in the queue.** `MAX_QUEUED=50 nohup bash .../submit_all_fixes.sh > fixes.log 2>&1 &`

**Stop submitting.** `pkill -f submit_all_fixes.sh`. Jobs already submitted keep running; cancel them with `scancel` if needed.

**Undo a repair.** Nothing is deleted. Cancel the material's jobs, move the files from `<folder>/stale_<date>/<step>/` back into `<step>/` and remove the `TAG = value` lines added at the end of the INCAR. `01_relax/POSCAR.orig` is the structure before the first restart.

**Problems in the log.** `awk '/^!!!|^-- /' fixes.log` lists failed submissions (`!!!`) and materials skipped because they had jobs in the queue or their outputs had changed since the diagnosis (`--`); for the latter, diagnose again.

## 5 Reference

### 5.1 Command line

```
python3 vasp_diagnose.py LIST [options]
```

| Option | Default | Meaning |
|---|---|---|
| `--out DIR` | `vasp_diagnose_<date>_<time>` | folder for the report and the scripts |
| `--root DIR` | — | another folder to look up material names in (can be repeated) |
| `--since DATE` | 14 days ago | how far back SLURM's job records (`sacct`) are read, YYYY-MM-DD; must reach back to the jobs being diagnosed |
| `--kpar-max N` | 8 | largest KPAR written by the out-of-memory repair |
| `--spill-max X` | 5 | LOBSTER charge spilling (%) above which a material is flagged |
| `--retry-unknown` | off | also submit the materials whose failure was not recognised |
| `--max-queued N` | 200 | default queue limit of `submit_all_fixes.sh` (override later with `MAX_QUEUED=`) |

It needs only Python ≥ 3.6 (no extra packages) and uses `sacct` and `squeue` when they exist.

## 5.2 The list file

One entry per line; empty lines, `#` comments and anything after the first word are ignored. An entry is

| Entry | Checked |
|---|---|
| a **material name** — any folder name: `mp-149`, `GaAs`, `Bi2Se3_soc`, … | the folders of that name in $R$, $R$`/*/` or $R$`/*/materials/`, where $R$ is the folder holding the list file, and each `--root` |
| a **material or project folder** (has `00_convergence`, `01_relax`, …) | that folder |
| a **high-throughput run folder** (has `materials/`) | every material in `materials/` |
| any **other folder** | every sub-folder that is a material/project folder |

A word that is a folder (relative to the list file's folder) is taken as a path; otherwise it is a material name. Names are matched exactly, so `GaAs` finds `.../GaAs` but not `.../GaAs_PBE`, and a name may contain any characters a folder name can. A name that is found nowhere is reported as `missing`.

## 5.3 Kinds of runs

How the jobs are resubmitted depends on how the run was made:

| Layout | Recognised by | Resubmitted with |
|---|---|---|
| `ht` | `job.sbatch` in the material folder (high-throughput drivers) | `sbatch job.sbatch <step>`, walltimes from `TIME_*` in the run's `env.sh`; steps relax, scf, bands, lobster |
| `slurm` | `run.sh` files with `#SBATCH` lines (`vasp-agent-slurm.py`) | `sbatch <step>/run.sh`, with its job name and walltime |
| `local` | anything else (`vasp-agent.py`) | `(cd <step> && bash run.sh)`, in the foreground |

Each material gets a *label*, `<run folder>__<material>` (e.g. `HT_semimetals__mp-149`, `projects__GaAs_test`); it names the repair script and the report line.

### 5.4 When is a step OK?

| Step | OK when |
|---|---|
| `00_convergence` | every point of the ENCUT test (`encut/encut_<E>/`) and of the k-point test (`kpoints/<N1>x<N2>x<N3>/`) that its `run.sh` will make has finished with a converged SCF. If `converged_params.txt` says a quantity did not converge in the tested range, this is noted in the report (widen the range yourself) |
| `01_relax` | OUTCAR has `reached required accuracy` |
| `02_scf` | OUTCAR finished (`General timing and accounting`) and the SCF converged (`aborting loop because EDIFF is reached`) |
| `03_bands` | finished, converged, and no energy above $10^6$ eV in OSZICAR |
| `04_dos` | as bands, and a non-empty DOSCAR |
| `06_dfpt` | finished with Born charges or the dielectric tensor in OUTCAR |
| `08_lobster` | `lobster.out` has `finished in`; a charge spilling above `--spill-max` is reported but counts as OK |

`05_wannier` and `07_phonons` are not checked (noted as `unchecked`).

For a step that is not OK, the cause is read from the step's newest SLURM logs, `vasp.out`, the end of OUTCAR and the SLURM job record (state, run time, time limit).

## 5.5 Failure causes and their repairs

| Name | Meaning | Repair |
|---|---|---|
| `oom` | out of memory (`insufficient virtual memory`, `Cannot allocate memory`, killed by the OOM handler, MPI time-outs between nodes) | KPAR lowered to at most `--kpar-max` (a divisor of the MPI ranks); all node memory requested |
| `timeout` | the job hit its time limit | walltime doubled; a relax continues from its last structure |
| `scf_nelm` | SCF stopped at NELM without converging | `ALGO = All`, `NELM` $\geq 150$ |
| `nscf_diverged` | bands/DOS/LOBSTER energies exploded | 30% more bands than the SCF, `ALGO = Normal`, `NELM` $\geq 150$ |
| `nscf_unconv` | bands/DOS stopped at NELM | `ALGO = Normal`, `NELM` $\geq 150$ |
| `relax_unconv` | relax ended without reaching the force criterion | continues from CONTCAR (or the last XDATCAR frame) |
| `zbrent` | `ZBRENT: fatal error` in a relax | `IBRION` $2 \rightarrow 1$, continue |
| `symmetry` | VASP symmetry errors | `SYMPREC = 1E-4` (`ISYM = 0` if SYMPREC was set) |
| `edddav` | `EDDDAV/ZHEGV` errors | SCF: `ALGO = All`; relax: `ALGO = Normal`; bands etc.: as nscf_diverged |
| `wavecar` | WAVECAR does not fit the new run | WAVECAR moved aside, start from CHGCAR |
| `chgcar` | CHGCAR unreadable or cut off | SCF started from scratch (`ICHARG = 2`) |
| `chgcar_missing` | a later step needs `02_scf/CHGCAR`, which is missing or cut off | SCF redone first |
| `lobster_fail` | VASP part of the LOBSTER step finished, the LOBSTER program did not | only the LOBSTER program is rerun |
| `quota` | disk full / quota exceeded | resubmitted unchanged — **free space first** |
| `mpi_init` | MPI failed to start (bad node) | resubmitted unchanged |
| `not_run` | never ran (e.g. an earlier step failed) | submitted |
| `unknown` | none of the above | script written but held back for review |
| `spilling_high` | LOBSTER finished, spilling > `--spill-max` | none (report only) |
| `missing` | a listed name has no folder | none (report only) |
| `phase2_pending` | convergence tests done, production steps not started | none: start them with `submit_calculations.sh` / `run_calculations.sh` |
| `queued` | the material still has jobs in the queue | none: diagnose later |

A material is counted under the cause of its *first* failed step. A charge spilling is a property of the LOBSTER basis set, so rerunning does not change it; lowering it needs other basis functions in `lobsterin`.

## 5.6 What is redone

- If the relax or the SCF failed, that step and *all* later steps are redone.
- Otherwise only the failed later steps (bands, DOS, DFPT, LOBSTER). If they need `02_scf/CHGCAR` and it is missing or cut off, the SCF is redone first.

- **Convergence tests** are repaired on their own: only the broken points are moved aside, the fix of their cause goes into the test's INCAR, and the test's `run.sh` is resubmitted — it skips the points that are already done. Production steps are not redone because of a convergence test.
- **LOBSTER only**: when the VASP part of the LOBSTER step finished and its WAVECAR is there, only the LOBSTER program reruns (not for local projects).
- In high-throughput runs `job.sbatch` cannot run DOS or DFPT; such steps are reported (`job.sbatch cannot run: dos`) but not resubmitted.

What a repair script changes in a material folder:

- **INCAR**: every line of a changed tag is removed and `TAG = value` is added at the end. No other tags are touched.
- **Old outputs** of every redone step (OUTCAR, OSZICAR, CHGCAR, WAVECAR, `*.lobster`, ..., or the whole broken convergence point) are moved to `<folder>/stale_<date>/`. Inputs stay in place; nothing is deleted.
- **Relax restarts** from the newest complete CONTCAR (else the last XDATCAR frame); the original POSCAR is kept once as `POSCAR.orig`.
- **SCF start**: if the relax's CHGCAR is cut off, the SCF starts from scratch.

## 5.7 Order of the resubmitted jobs

| Layout | Order |
|---|---|
| `ht` | each step waits for the previous one to succeed (`afterok`); LOBSTER after bands waits only for it to end (`afterany`), as it does not need the bands |
| `slurm` | SCF waits for the relax; bands, DOS, DFPT and LOBSTER wait for the SCF |
| `local` | one after the other; stops at the first failure |

Convergence tests are submitted without waiting for anything. Walltimes are the run's own (`env.sh`, `#SBATCH --time`, or relax 8 h, SCF 4 h, bands/DOS 2 h, DFPT 8 h, LOBSTER 4 h, tests 8 h), doubled after a time-out.

## 5.8 Output files

`diagnosis.tsv` — tab-separated, one line per material:

| Column | Content |
|---|---|
| `label` | the material's label |
| `layout` | `ht`, `slurm`, `local` (`-` for a missing name) |
| `path` | its folder |
| `steps` | `step:status` for each step (`conv:encut`, `conv:kpoints`, `relax`, ...) and notes |
| `action` | `ok`, a report-only name, or `<cause> -> <steps redone>`; `REVIEW` in front when held back |
| `script` | its repair script, if any |

```
awk -F'\t' 'NR>1␣{split($5,a,"␣");␣n[a[1]]++}␣END␣{for␣(k␣in␣n)␣
    print␣k,␣n[k]}' diagnosis.tsv
awk -F'\t' '$5␣~␣/^REVIEW/␣{print␣$1}' diagnosis.tsv
```

**fix/<label>.sh** — a stand-alone bash script; it can be run on its own. In order it:

1. records the diagnosis and the error lines in its header;
2. **stops** if the material has jobs in the queue, or if any of its OUTCARs or `lobster.out` changed since the diagnosis (so it is safe to run late, or twice);
3. applies the INCAR changes and moves the old outputs aside;
4. submits the jobs (or runs them, for a local project);
5. leaves a marker `.submitted/<label>` in the output folder.

**submit_all_fixes.sh** — runs every repair script in turn; skips those with a `.submitted` marker; waits while you have `MAX_QUEUED` or more jobs in the queue (all your jobs count); lists the held-back `unknown` scripts at the end as commented-out lines.

## 5.9 Limitations

- Jobs are recognised by their job names (`<material>_<step>`, or the name in the step's `run.sh`). Jobs submitted under other names are not seen, and a failure cause is then taken from the log files only.
- Only jobs since `--since` are known to the tool. Without a job record a time-out is still recognised from the log, but the walltime is doubled from the configured value.
- A second out-of-memory failure with KPAR already $\leq$ `--kpar-max` is resubmitted with the same KPAR; diagnose again with a smaller `--kpar-max`.
- A material that fails again with the same cause after one repair (e.g. `scf_nelm`) needs a look by hand.
- `quota` and `mpi_init` are resubmitted unchanged: free disk space first; node failures are usually transient.
- `05_wannier` and `07_phonons` are not checked; a convergence-test range that was too small is reported but not widened.
- Repair scripts use GNU `sed` and `stat`: they are meant for Linux clusters.